



**SORBONNE
UNIVERSITY**

Faculté des Sciences et Ingénierie

Parcours Science et Technologie du Logiciel (STL-INSTA)

TPALT

Rapport de projet SORBOT

Étudiants :

BOUAFIA Merouane, N°21501802, merouane.bouafia@etu.sorbonne-universite.fr

SADI Aksil, N°21427293, aksil.sadi@etu.sorbonne-universite.fr

VILAYVANH Mickael, N°21310046, mickael.vilayvanh@etu.sorbonne-universite.fr

HALFAOUI Anis, N°21501595, anis.halfaoui@etu.sorbonne-universite.fr

Mars 2026

1 Introduction

Sorbot transforme des donnees de marche en signaux de trading, puis orchestre leur validation et leur execution. Le projet est pense pour un cadre pedagogique mais reprend des pratiques proches d'une architecture de production : separation en services, API explicites, persistance des evenements metier et supervision temps reel.

Les objectifs sont les suivants : produire des signaux exploitables, appliquer des regles de risque claires, et offrir une interface de controle en mode manuel ou automatique.

2 Architecture et stack technique

L'application repose sur trois services :

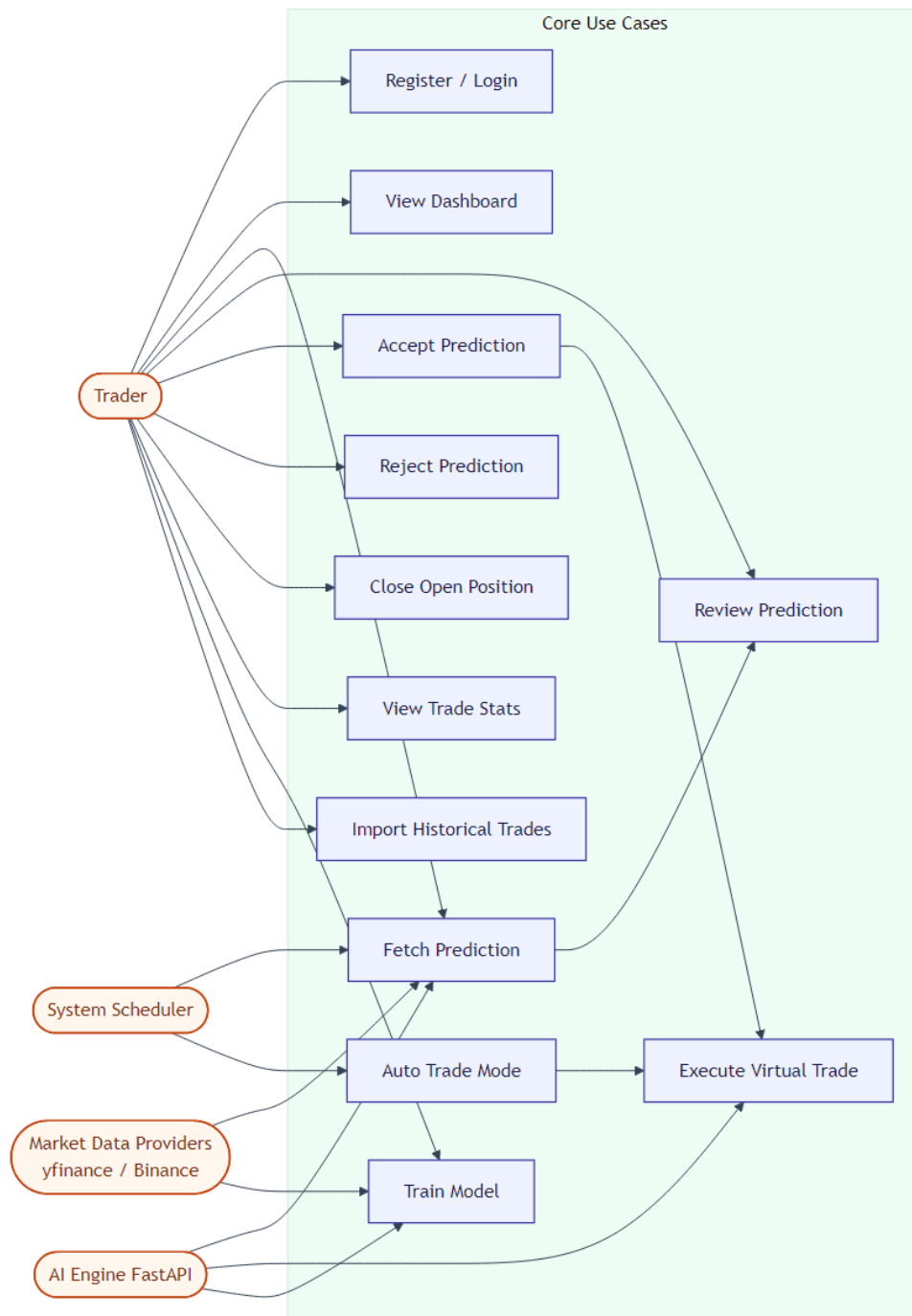
- **Frontend** React/Vite (port 3000),
- **Backend** Spring Boot (port 8081),
- **AI Engine** FastAPI/XGBoost (port 8000).

Le backend joue le role d'orchestrateur : il recoit les demandes du frontend, interroge l'AI Engine, persiste predictions et trades, puis diffuse les changements via WebSocket.

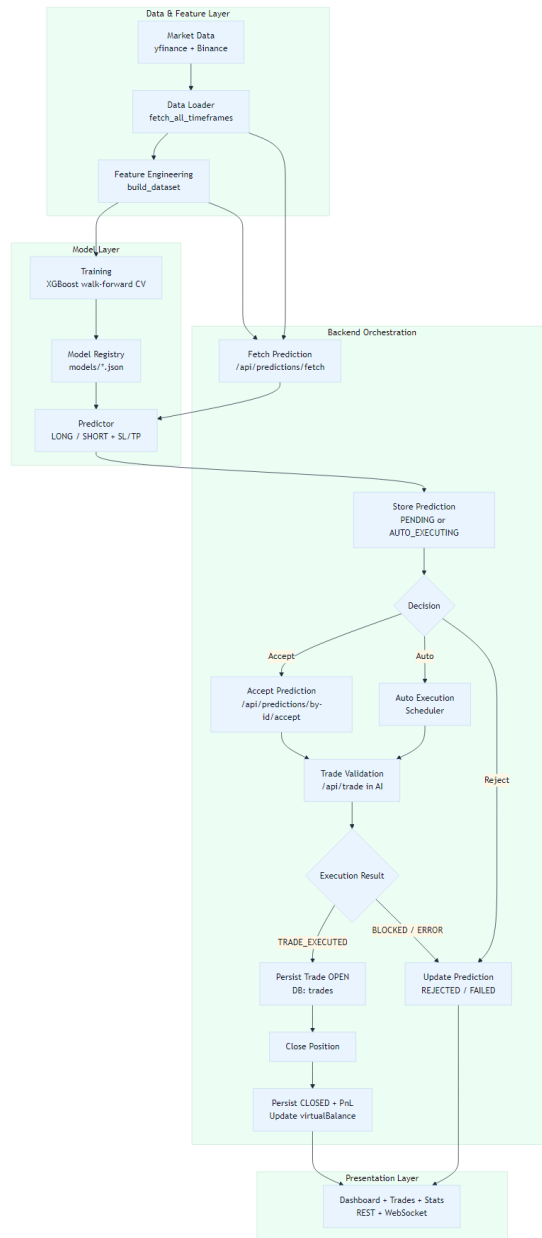
TABLE 1 – Vue synthetique des services

Service	Technologies	Responsabilite principale
AI Engine	Python 3.11, FastAPI, XGBoost	Entrainement, prediction, execution de trade
Backend	Java 17, Spring Boot, PostgreSQL	API, orchestration, persistance, scheduler
Frontend	React 19, Vite, STOMP/SockJS	Supervision, pilotage, visualisation

3 Diagrammes



(a) Cas d'usage



(b) Flux données-trade

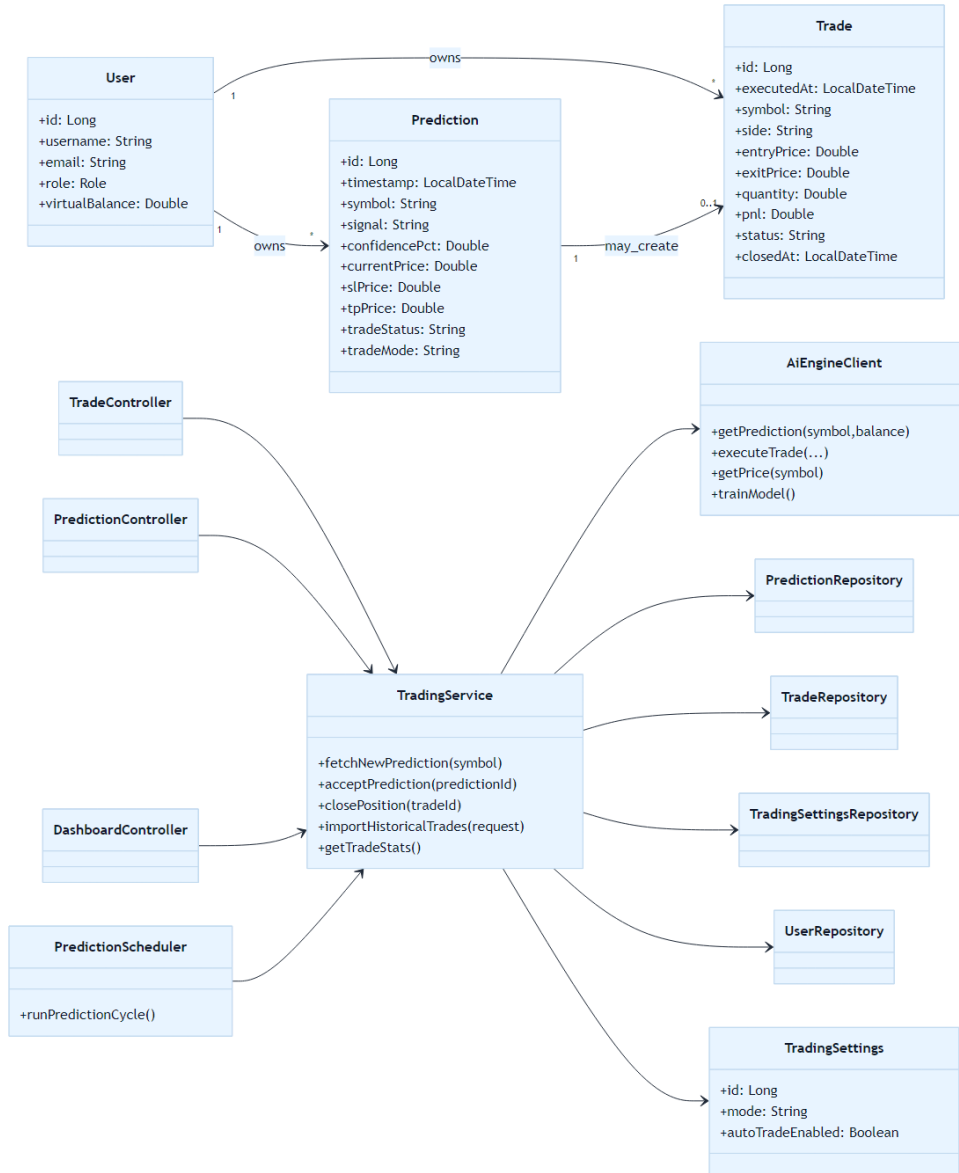


FIGURE 1 – Synthèse visuelle de l’architecture, des usages métier et de la structure logicielle.

4 Flux fonctionnel prediction-execution

Le cycle métier suit cinq étapes :

1. collecte de données de marché et calcul des indicateurs,
2. prediction (LONG, SHORT, NO_TRADE) par l’AI Engine,
3. enregistrement backend avec statut initial (PENDING ou AUTO_EXECUTED),
4. décision utilisateur (mode MANUAL) ou execution automatique (mode AUTO),
5. mise à jour des trades/statistiques et diffusion temps réel vers le frontend.

Le mode **MANUAL** privilégie le contrôle humain (acceptation/rejet), alors que le mode **AUTO** vise la rapidité d’exécution selon des règles strictes.

5 Pipeline IA et cycle de vie du modèle

Le moteur IA suit une chaîne de traitement cohérente :

1. acquisition multi-horizons (1h, 4h, 1d),
2. génération de 122+ features strictement lagées,
3. apprentissage XGBoost avec validation walk-forward,

4. inference enrichie (SL/TP, regime, indicateurs),
5. reentrainement periodique avec validation avant remplacement.

La strategie d'inference applique des seuils explicites :

- LONG si probabillite $\geq 0,65$,
- SHORT si probabillite $\leq 0,35$ (bloque en spot),
- NO_TRADE dans la zone intermediaire.

Le scheduler de retrain tourne periodiquement (par default toutes les 6 heures), compare le nouveau modele a l'ancien, puis n'accepte le remplacement que si la degradation de qualite reste sous un seuil controle. Ce mecanisme limite les regressions en phase de marche instable.

6 Execution de trade et gestion du risque

La politique de risque est volontairement conservative :

- risque par trade limite (environ 1,5% du capital),
- plafond de positions ouvertes (une a la fois),
- stop-loss / take-profit derives de l'ATR,
- rejet des opportunités avec ratio rendement/risque insuffisant,
- verification des soldes minimums avant execution.

En spot, le flux standard est *market buy* puis ordre OCO (TP+SL), avec fallback vers un stop separe si necessaire. Le backend journalise tous les etats metier (prediction, trade, cloture, erreurs), ce qui facilite l'audit et l'analyse de performance.

7 Forces, limites et recommandations

Forces :

- architecture claire et decouplee,
- pipeline IA riche et coherent avec les contraintes temporelles,
- regles de risque explicites,
- supervision temps reel via dashboard + WebSocket.

Limites :

- sensibilite a la qualite des donnees externes,
- risque de derive des performances selon le regime de marche,
- besoin de renforcer observabilite et tests d'integration,
- seuils de decision encore majoritairement fixes.

Actions recommandees a court terme :

1. ajouter des tests bout-en-bout prediction-execution-persistence,
2. monitorer le drift de donnees et la calibration du modele,
3. introduire des garde-fous operationnels (limites journalieres, circuit breaker).

8 Conclusion

Sorbot propose une base serieuse pour l'apprentissage du trading algorithmique moderne : architecture modulaire, circuit metier explicite, et politique de risque deja formalisee. Pour atteindre un niveau plus industriel, la priorite est de consolider la chaine de qualite (tests, monitoring, alerting) et d'ameliorer l'adaptativite des decisions en fonction des regimes de marche.